

**Proyek 3 Proyek Pengembangan Perangkat Lunak Berbasis Web****Modul 3****Frameworks in Programming - Laravel Basic**

## Daftar Isi

## Daftar Isi

|                                                              |    |
|--------------------------------------------------------------|----|
| Daftar Isi.....                                              | 2  |
| 1. Modul Overview .....                                      | 4  |
| 1.1 Tujuan Pembelajaran .....                                | 4  |
| 1.2 Materi Esensial.....                                     | 4  |
| 1.3 Target Milestones .....                                  | 5  |
| 1.4 Pembagian Materi.....                                    | 5  |
| 1.5 Alur Belajar yang Digunakan .....                        | 5  |
| 1.6 Prasyarat dan Baseline Lingkungan .....                  | 5  |
| 2. Kasus Terintegrasi: Aplikasi Manajemen Kegiatan .....     | 6  |
| 2.1 Lingkup Modul 3 .....                                    | 6  |
| 2.2 Data Kegiatan.....                                       | 6  |
| 2.3 Business Rules Modul 3 .....                             | 6  |
| 2.4 Acceptance Criteria Modul 3 .....                        | 6  |
| 2.5 Peta Tanggung Jawab .....                                | 7  |
| 3. Fondasi Laravel dan Request Lifecycle .....               | 8  |
| 3.1 Dari Halaman Statis ke Aplikasi Server-Side .....        | 8  |
| 3.2 Membuat Proyek Laravel .....                             | 8  |
| 3.3 Struktur Direktori yang Perlu Dikenali .....             | 8  |
| 3.4 Request Lifecycle Laravel .....                          | 8  |
| 3.5 Routing.....                                             | 9  |
| 3.6 Controller sebagai Orkestrator.....                      | 9  |
| 3.7 Blade: Menyajikan Response HTML .....                    | 9  |
| 3.8 Database dan Migration .....                             | 10 |
| 3.9 Eloquent Model .....                                     | 10 |
| 3.10 Data Awal untuk Observasi .....                         | 11 |
| 4. CRUD, Business Logic, dan Clean Code.....                 | 11 |
| 4.1 Resource Route dan Resource Controller .....             | 11 |
| 4.2 Route Model Binding .....                                | 11 |
| 4.3 Form Request sebagai Boundary Input .....                | 12 |
| 4.4 Controller yang Fokus pada Orkestrasi .....              | 12 |
| 4.5 Service Class dan Business Logic.....                    | 13 |
| 4.6 Form Blade dan Error Validation.....                     | 13 |
| 4.7 Clean Code dalam Konteks Laravel .....                   | 14 |
| 4.8 Menulis Spesifikasi yang Dapat Dievaluasi .....          | 14 |
| 4.9 Static Analysis dengan SonarQube .....                   | 14 |
| 5. Eksperimen Praktikum .....                                | 15 |
| Eksperimen 1. Menelusuri Request Lifecycle Laravel .....     | 15 |
| Eksperimen 2. Mendiagnosis Fatal Controller.....             | 15 |
| Eksperimen 3. Business Rule di Tempat yang Salah .....       | 16 |
| Eksperimen 4. Static Analysis dan Refaktorisasi Terarah..... | 16 |
| 6. Tugas Praktikum .....                                     | 16 |

|                                                               |    |
|---------------------------------------------------------------|----|
| Task 1. Laravel Skeleton dan Request Lifecycle .....          | 16 |
| Task 2. Activity CRUD dengan Form Request .....               | 17 |
| Task 3. Business Logic Refactor dan Quality Baseline.....     | 19 |
| 7. Independent Challenge dan Homework .....                   | 21 |
| 7.1 Independent Challenge – Filter Kegiatan .....             | 21 |
| 7.2 Homework – Architecture Review dan Refactor Terarah ..... | 21 |
| 8. Checkpoint Integrasi Modul 3 – Activity Manager v1.....    | 21 |
| 8.1 Definition of Done.....                                   | 21 |
| 8.2 Struktur Repository Minimum .....                         | 21 |
| 8.3 Checklist Repository dan GitHub .....                     | 22 |
| 9. Penilaian dan Refleksi .....                               | 22 |
| 9.1 Self Assessment.....                                      | 22 |
| 9.2 Peer Assessment.....                                      | 22 |
| 9.3 Lesson Learnt.....                                        | 22 |
| 10. Rubrik Penilaian Dosen.....                               | 23 |
| 11. Referensi.....                                            | 23 |

# 1. Modul Overview

Pada Modul 1 mahasiswa membangun struktur dan presentasi halaman menggunakan HTML dan CSS. Pada Modul 2 mahasiswa menambahkan perilaku menggunakan Vanilla JavaScript, DOM, event, validasi, asynchronous JavaScript, dan debugging. Modul 3 memindahkan perhatian dari satu halaman di browser menuju aplikasi web server-side yang memiliki lifecycle request, routing, controller, view, persistence, validasi, dan aturan bisnis yang harus ditempatkan secara sadar.

Tujuan utama modul ini bukan sekadar agar mahasiswa dapat membuat CRUD Laravel. Mahasiswa harus memahami bagaimana sebuah request diproses, bagaimana tanggung jawab dipisahkan, di mana business rule seharusnya ditempatkan, dan bagaimana spesifikasi fitur digunakan sebagai dasar evaluasi kualitas kode.

## 1.1 Tujuan Pembelajaran

1. Menjelaskan alur request Laravel dari public/index.php, bootstrap aplikasi, middleware, router, controller, proses aplikasi, sampai response dikirim kembali ke browser.
2. Membedakan tanggung jawab route, controller, Form Request, service class, Eloquent model, migration, dan Blade view.
3. Membangun fitur daftar, detail, tambah, ubah, dan hapus kegiatan menggunakan pola resource Laravel.
4. Menggunakan migration dan Eloquent untuk merepresentasikan struktur data tanpa menulis SQL untuk operasi CRUD dasar.
5. Menggunakan Form Request untuk memvalidasi input pada boundary HTTP dan membedakannya dari business rule.
6. Menerapkan route model binding untuk memperoleh model berdasarkan parameter route dengan kode yang lebih jelas.
7. Memindahkan business logic yang layak dipisahkan dari controller ke service class sehingga controller berfokus pada orkestrasi request dan response.
8. Menerapkan prinsip clean code pada penamaan, ukuran method, duplikasi, guard clause, dan separation of concerns.
9. Menulis business rules dan acceptance criteria yang eksplisit serta dapat diuji kembali pada modul pengujian.
10. Menggunakan SonarQube untuk mengidentifikasi code smell dan mendokumentasikan keputusan refaktorisasi berdasarkan bukti.
11. Menjelaskan seluruh keputusan teknis yang dibuat serta melakukan perubahan kecil terhadap kode tanpa bergantung pada AI.

## 1.2 Materi Esensial

| Konsep                 | Makna yang Harus Dipahami                                                                                                                                                                    |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework              | Laravel menyediakan struktur, konvensi, dan komponen yang mempercepat pengembangan; framework tidak otomatis menjamin arsitektur yang baik.                                                  |
| Request lifecycle      | Setiap request masuk melalui entry point aplikasi, melewati bootstrap dan middleware, dipetakan oleh router, diproses handler/controller, lalu menghasilkan response.                        |
| Separation of concerns | Route memetakan request, controller mengorkestrasi, Form Request memvalidasi input, service menampung business rule yang layak dipisahkan, Eloquent menangani data, Blade menyajikan output. |
| Persistence            | Migration mendefinisikan perubahan skema; Eloquent model merepresentasikan data dan operasi yang terkait dengan entitas.                                                                     |
| Business rule          | Aturan domain tidak boleh tersebar di Blade dan controller. Aturan harus memiliki lokasi yang dapat dijelaskan dan, bila perlu, diuji terpisah.                                              |
| Clean code             | Kode harus mudah dibaca, memiliki penamaan jelas, tanggung jawab terbatas, duplikasi minimum, dan alur yang dapat dijelaskan.                                                                |
| Specification          | Acceptance criteria mendeskripsikan perilaku yang harus dapat dibuktikan, bukan hanya daftar fitur yang diharapkan.                                                                          |
| Static analysis        | SonarQube membantu menemukan indikasi code smell, duplication, complexity, maintainability issue, dan masalah kualitas lain; hasilnya perlu ditafsirkan, bukan hanya dikejar skornya.        |

### 1.3 Target Milestones

| Milestone                | Indikator Keberhasilan                                                                                                      |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Framework Ready          | Proyek Laravel berjalan; mahasiswa dapat menunjukkan entry point, route, controller, dan Blade yang menangani satu request. |
| Data Ready               | Migration berhasil dijalankan; Eloquent model dapat membaca dan menyimpan data kegiatan.                                    |
| CRUD Ready               | Resource route/controller dan Form Request menangani CRUD normal case dan input tidak valid.                                |
| Business Logic Separated | Aturan transisi status tidak berada di Blade; controller tidak memuat seluruh aturan bisnis.                                |
| Specification Ready      | Business rules dan acceptance criteria tertulis dan digunakan untuk memeriksa implementasi.                                 |
| Quality Baseline         | Static analysis dijalankan, satu temuan relevan dianalisis, dan satu refaktorisasi dapat dijelaskan.                        |

### 1.4 Pembagian Materi

| Level    | Fokus                                | Aktivitas Utama                                                                                            | Output                                                                                     |
|----------|--------------------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Basic    | Fondasi Laravel dan lifecycle        | Setup proyek; struktur folder; request lifecycle; route; controller; Blade; migration; Eloquent; Task 1.   | Aplikasi dapat menampilkan daftar dan detail kegiatan dari database.                       |
| Practice | CRUD dan business logic architecture | Resource CRUD; Form Request; route model binding; service class; acceptance criteria; SonarQube; Task 2–3. | CRUD teruji, business rules eksplisit, controller tetap fokus, baseline kualitas tersedia. |

### 1.5 Alur Belajar yang Digunakan

| Tahap   | Aktivitas Mahasiswa                                                               | Bukti                             |
|---------|-----------------------------------------------------------------------------------|-----------------------------------|
| Predict | Tuliskan dugaan alur atau output sebelum menjalankan perintah/kode.               | Prediksi pada worksheet.          |
| Run     | Jalankan satu perubahan kecil yang dapat diamati.                                 | Kode, route, query, atau request. |
| Observe | Periksa response, route:list, halaman, database, log, atau hasil static analysis. | Bukti aktual.                     |
| Explain | Jelaskan mengapa hasil tersebut muncul dan komponen mana yang bertanggung jawab.  | Penjelasan dengan kata sendiri.   |
| Improve | Perbaiki kode berdasarkan bukti dan uji kembali acceptance criteria.              | Versi akhir dan commit.           |

#### Aturan penggunaan AI

Mahasiswa wajib membuat prediksi, struktur awal, acceptance criteria, dan percobaan diagnosis sendiri. AI hanya digunakan untuk meminta penjelasan konsep, memperoleh hint bertahap, atau meninjau edge case setelah terdapat bukti percobaan. AI tidak digunakan untuk menghasilkan satu task secara utuh. Seluruh bantuan dicatat pada worksheet.

### 1.6 Prasyarat dan Baseline Lingkungan

- 1) PHP 8.3 atau lebih baru dan Composer tersedia pada komputer.
- 2) Git tersedia dan mahasiswa memiliki repository GitHub yang akan dipakai kembali sampai Modul 6.
- 3) Laravel 13 digunakan sebagai baseline kelas.
- 4) Database baseline menggunakan SQLite agar setup seragam. Jika kelas menggunakan MySQL/MariaDB, dosen memberikan konfigurasi .env yang sama untuk seluruh mahasiswa.
- 5) SonarQube Server/Community Build atau endpoint kelas disiapkan oleh dosen/laboratorium; mahasiswa tidak dinilai dari kemampuan mengadministrasikan server SonarQube.
- 6) Editor/IDE dapat menjalankan terminal dan membaca file PHP dengan nyaman.

## 2. Kasus Terintegrasi: Aplikasi Manajemen Kegiatan

Mulai Modul 3 sampai Modul 6 digunakan kasus yang sama, yaitu Aplikasi Manajemen Kegiatan. Domain sengaja dibuat sederhana agar perhatian utama tetap berada pada arsitektur, business rules, state, authorization, logging, maintainability, dan pengujian. Pada Modul 3 versi aplikasi disebut Activity Manager v1.

### 2.1 Lingkup Modul 3

- 1) Melihat daftar kegiatan.
- 2) Melihat detail kegiatan.
- 3) Menambah kegiatan baru.
- 4) Mengubah kegiatan.
- 5) Menghapus kegiatan.
- 6) Memfilter daftar berdasarkan status atau kategori sebagai independent challenge.
- 7) Mengelola nilai status Planned, Ongoing, dan Done.
- 8) Menerapkan aturan transisi status sederhana sebagai contoh business logic yang tidak diletakkan pada Blade.

#### Batas modul

Authentication, session, ownership pengguna, dan role belum diimplementasikan pada Modul 3. Komponen tersebut ditambahkan pada Modul 4 dan Modul 5. Pada Modul 3, fokusnya adalah struktur framework, CRUD, business logic, spesifikasi, dan kualitas kode.

### 2.2 Data Kegiatan

| Field                   | Tipe Konseptual       | Keterangan                     |
|-------------------------|-----------------------|--------------------------------|
| id                      | integer / primary key | Dibuat otomatis oleh database. |
| title                   | string, maksimum 100  | Judul kegiatan.                |
| description             | text, nullable        | Deskripsi ringkas.             |
| activity_date           | date                  | Tanggal kegiatan.              |
| category                | string, maksimum 50   | Kategori kegiatan.             |
| status                  | string, maksimum 20   | Planned, Ongoing, atau Done.   |
| created_at / updated_at | timestamp             | Dikelola oleh Laravel.         |

### 2.3 Business Rules Modul 3

| ID     | Aturan                                                                                                                   | Lokasi / Waktu Implementasi       |
|--------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| BR-01  | Judul kegiatan wajib diisi, bertipe teks, dan panjangnya 5–100 karakter.                                                 | Boundary input / Form Request.    |
| BR-02  | Tanggal kegiatan wajib diisi dan harus berupa tanggal yang valid.                                                        | Boundary input / Form Request.    |
| BR-03  | Status hanya dapat bernilai Planned, Ongoing, atau Done.                                                                 | Boundary input + domain state.    |
| BR-03A | Transisi status berjalan maju: Planned → Ongoing → Done. Status tidak boleh mundur setelah bergerak ke tahap berikutnya. | Business logic pada service.      |
| BR-04  | Member hanya dapat mengubah/menghapus kegiatan miliknya.                                                                 | Belum diimplementasikan; Modul 5. |
| BR-05  | Admin dapat mengelola seluruh kegiatan.                                                                                  | Belum diimplementasikan; Modul 5. |
| BR-06  | Data keamanan tidak disimpan pada localStorage.                                                                          | Belum diimplementasikan; Modul 4. |
| BR-07  | Aksi penting dicatat pada log.                                                                                           | Diperkenalkan penuh pada Modul 4. |

### 2.4 Acceptance Criteria Modul 3

| ID    | Fitur           | Kondisi Diterima                                                                                                                     |
|-------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------|
| AC-01 | Daftar kegiatan | Ketika membuka /activities, seluruh kegiatan tampil tanpa error danurut berdasarkan tanggal terbaru/terdekat sesuai keputusan kelas. |
| AC-02 | Detail kegiatan | Ketika membuka route detail dengan model yang valid, data kegiatan tampil; model yang tidak ditemukan menghasilkan respons 404.      |

| ID    | Fitur                       | Kondisi Diterima                                                                                                                            |
|-------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| AC-03 | Tambah valid                | Input valid menghasilkan tepat satu data baru dan pengguna diarahkan ke halaman yang ditentukan dengan pesan berhasil.                      |
| AC-04 | Judul terlalu pendek        | Judul kurang dari 5 karakter ditolak dan pesan validasi tampil dekat input.                                                                 |
| AC-05 | Status tidak valid          | Status di luar Planned/Ongoing/Done ditolak oleh server.                                                                                    |
| AC-06 | Ubah valid                  | Perubahan field valid tersimpan dan data lama tidak terduplikasi.                                                                           |
| AC-07 | Hapus                       | Penghapusan satu kegiatan menghilangkan tepat data tersebut dan tidak memengaruhi data lain.                                                |
| AC-08 | Transisi status valid       | Planned dapat menjadi Ongoing dan Ongoing dapat menjadi Done.                                                                               |
| AC-09 | Transisi status tidak valid | Done tidak dapat kembali ke Ongoing/Planned; aplikasi memberi pesan yang dapat dipahami.                                                    |
| AC-10 | Kualitas                    | Tidak ada error aplikasi pada alur normal; route/controller/view tidak memuat duplikasi yang tidak perlu; static analysis telah dijalankan. |

## 2.5 Peta Tanggung Jawab

| Komponen       | Tanggung Jawab                                                                               | Batas                                                          |
|----------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| routes/web.php | Memetakan method + URL ke controller/handler.                                                | Jangan menyimpan proses CRUD panjang di route closure.         |
| Controller     | Menerima dependency, mengorkestrasi request, memanggil service/model, memilih response/view. | Jangan menjadi tempat seluruh validasi dan business rule.      |
| Form Request   | Memeriksa bentuk/input request dan menyiapkan data tervalidasi.                              | Tidak menggantikan semua business rule domain.                 |
| Service        | Menampung business logic yang layak dipisahkan dan dapat dijelaskan sebagai aturan aplikasi. | Jangan menjadi kumpulan function tanpa tanggung jawab jelas.   |
| Eloquent Model | Merepresentasikan entitas data dan operasi Eloquent yang relevan.                            | Jangan mengisi model dengan seluruh logika UI/HTTP.            |
| Migration      | Mendefinisikan perubahan skema database secara versioned.                                    | Tidak berfungsi sebagai validasi request.                      |
| Blade          | Menyajikan data dan state tampilan.                                                          | Tidak menjadi lokasi utama business rule atau keputusan akses. |
| SonarQube      | Mengobservasi indikasi kualitas kode secara statis.                                          | Tidak membuktikan bahwa fitur benar secara fungsional.         |

### 3. Fondasi Laravel dan Request Lifecycle

#### 3.1 Dari Halaman Statis ke Aplikasi Server-Side

Pada Modul 1 dan Modul 2, browser memuat HTML/CSS/JavaScript lalu JavaScript memanipulasi DOM di sisi klien. Pada Laravel, browser tetap menjadi client, tetapi request diarahkan ke aplikasi server. Server menentukan route, menjalankan proses aplikasi, berinteraksi dengan database bila diperlukan, kemudian mengirim response. Blade menghasilkan HTML di server sebelum diterima browser.

##### Model mental

Browser tidak “menjalankan Laravel”. Browser mengirim HTTP request. Laravel berjalan pada server, memproses request, lalu menghasilkan HTTP response. Setelah response HTML diterima, browser kembali melakukan tugas yang telah dipelajari pada Modul 1–2: membentuk DOM, menerapkan CSS, dan menjalankan JavaScript bila ada.

#### 3.2 Membuat Proyek Laravel

##### Perintah dasar

```
composer create-project laravel/laravel activity-manager "13.*"
cd activity-manager
php artisan serve
```

Buka alamat yang ditampilkan oleh Artisan. Bila halaman awal Laravel tampil, jangan langsung melanjutkan. Catat URL, status response pada Network, dan folder proyek yang terbentuk.

**Pertanyaan pemahaman: Apa perbedaan tugas Composer, Artisan, dan web server pengembangan pada langkah di atas?**

#### 3.3 Struktur Direktori yang Perlu Dikenali

| Lokasi               | Peran                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------|
| app/Http/Controllers | Controller aplikasi.                                                                                     |
| app/Http/Requests    | Form Request buatan aplikasi.                                                                            |
| app/Models           | Eloquent model.                                                                                          |
| app/Services         | Service class buatan kita; folder ini bukan keharusan framework tetapi digunakan sebagai konvensi kelas. |
| bootstrap/app.php    | Bootstrap/configuration aplikasi.                                                                        |
| database/migrations  | Riwayat perubahan skema database.                                                                        |
| database/seeders     | Data awal/seed.                                                                                          |
| resources/views      | Blade views.                                                                                             |
| routes/web.php       | Route aplikasi web.                                                                                      |
| public/index.php     | Entry point request web.                                                                                 |
| storage              | Log, cache, session/file tertentu sesuai driver.                                                         |
| .env                 | Konfigurasi environment lokal; tidak dikomit ke repository.                                              |

#### 3.4 Request Lifecycle Laravel

Secara konseptual, request masuk melalui public/index.php. File tersebut memuat autoloader Composer dan memperoleh instance aplikasi dari bootstrap/app.php. Request kemudian diproses oleh HTTP kernel, melewati proses bootstrap dan middleware, diserahkan kepada router, kemudian route/controller menghasilkan response. Response berjalan kembali melalui middleware sebelum dikirim ke browser.

##### Model alur request untuk praktikum

```
Browser
↓ HTTP Request
public/index.php
↓ bootstrap/application
HTTP Kernel + Middleware
↓
```

```

Router (routes/web.php)
↓
Controller / Handler
↓
Application Logic + Eloquent
↓
Blade / Redirect / Response
↓
HTTP Response
↓
Browser

```

### Yang perlu diingat

Peta di atas adalah model belajar, bukan daftar semua class internal Laravel. Mahasiswa perlu mampu menjelaskan titik tanggung jawab yang relevan terhadap kode aplikasi, bukan menghafal seluruh internals framework.

## 3.5 Routing

routes/web.php

```

use App\Http\Controllers\ActivityController;
use Illuminate\Support\Facades\Route;

Route::get('/activities', [ActivityController::class, 'index'])
    ->name('activities.index');

Route::get('/activities/{activity}', [ActivityController::class, 'show'])
    ->name('activities.show');

```

Route menyatakan pasangan method HTTP, URI, handler, dan nama route. Nama route membantu view/controller menghasilkan URL tanpa menulis path secara berulang.

Mengobservasi route yang terdaftar

```
php artisan route:list
```

**Pertanyaan pemahaman: Mengapa nama route lebih stabil dibanding menulis string URL yang sama pada banyak tempat?**

## 3.6 Controller sebagai Orkestrator

app/Http/Controllers/ActivityController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Activity;
use Illuminate\View\View;

class ActivityController extends Controller
{
    public function index(): View
    {
        $activities = Activity::query()
            ->orderBy('activity_date')
            ->get();

        return view('activities.index', compact('activities'));
    }

    public function show(Activity $activity): View
    {
        return view('activities.show', compact('activity'));
    }
}

```

Controller di atas melakukan orkestrasi sederhana: mengambil data dan memilih view. Ia belum memuat validasi, aturan transisi status, atau markup HTML. Pemisahan ini menjadi fondasi ketika aplikasi tumbuh.

## 3.7 Blade: Menyajikan Response HTML

Contoh index Blade

```

{{-- resources/views/activities/index.blade.php --}}
@extends('layouts.app')

```

```

@section('content')
    <h1>Daftar Kegiatan</h1>

    @forelse ($activities as $activity)
        <article class="card">
            <h2>
                <a href="{{ route('activities.show', $activity) }}">
                    {{ $activity->title }}
                </a>
            </h2>
            <p>{{ $activity->activity_date->format('d M Y') }}</p>
            <p>Status: {{ $activity->status }}</p>
        </article>
    @empty
        <p>Belum ada kegiatan.</p>
    @endforelse
@endsection

```

Blade bertugas menyajikan data. Escape output dengan {{ ... }} untuk teks biasa. Jangan memindahkan aturan perubahan status, validasi inti, atau query database ke Blade.

### 3.8 Database dan Migration

Untuk baseline kelas, SQLite digunakan agar mahasiswa tidak menghabiskan waktu pada konfigurasi server database. Konsep migration dan Eloquent tetap sama bila kelas menggunakan MySQL/MariaDB.

#### Membuat model dan migration

```

php artisan make:model Activity -m
php artisan migrate

```

#### Potongan migration activities

```

Schema::create('activities', function (Blueprint $table) {
    $table->id();
    $table->string('title', 100);
    $table->text('description')->nullable();
    $table->date('activity_date');
    $table->string('category', 50);
    $table->string('status', 20)->default('Planned');
    $table->timestamps();
});

```

#### Konsep migration

Migration bukan file SQL biasa yang disalin untuk setiap komputer. Migration adalah riwayat perubahan skema yang ikut repository sehingga struktur database dapat dibentuk secara konsisten pada environment lain.

### 3.9 Eloquent Model

#### app/Models/Activity.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Activity extends Model
{
    protected $fillable = [
        'title',
        'description',
        'activity_date',
        'category',
        'status',
    ];

    protected function casts(): array
    {
        return [
            'activity_date' => 'date',
        ];
    }
}

```

Model di atas menentukan field yang boleh diisi melalui mass assignment dan mengubah `activity_date` menjadi object tanggal. Mahasiswa harus memahami bahwa model bukan sekadar “nama tabel dalam bentuk class”; model adalah representasi entitas yang menjadi pintu penggunaan Eloquent.

### 3.10 Data Awal untuk Observasi

#### Membuat seeder

```
php artisan make:seeder ActivitySeeder
```

#### Isi awal ActivitySeeder

```
Activity::query()->insert([
    [
        'title' => 'Workshop Git Dasar',
        'description' => 'Latihan kolaborasi repository.',
        'activity_date' => '2026-10-05',
        'category' => 'Workshop',
        'status' => 'Planned',
        'created_at' => now(),
        'updated_at' => now(),
    ],
    [
        'title' => 'Seminar Web Quality',
        'description' => 'Pengenalan maintainability dan testing.',
        'activity_date' => '2026-10-12',
        'category' => 'Seminar',
        'status' => 'Planned',
        'created_at' => now(),
        'updated_at' => now(),
    ],
]);
```

#### Menjalankan seeder

```
php artisan db:seed --class=ActivitySeeder
```

## 4. CRUD, Business Logic, dan Clean Code

### 4.1 Resource Route dan Resource Controller

#### Route resource

```
Route::resource('activities', ActivityController::class);
```

#### Membuat resource controller

```
php artisan make:controller ActivityController --resource --model=Activity
```

| Method    | URI                         | Method Controller | Tujuan           |
|-----------|-----------------------------|-------------------|------------------|
| GET       | /activities                 | index             | Daftar           |
| GET       | /activities/create          | create            | Form tambah      |
| POST      | /activities                 | store             | Simpan data baru |
| GET       | /activities/{activity}      | show              | Detail           |
| GET       | /activities/{activity}/edit | edit              | Form ubah        |
| PUT/PATCH | /activities/{activity}      | update            | Simpan perubahan |
| DELETE    | /activities/{activity}      | destroy           | Hapus            |

### 4.2 Route Model Binding

Ketika parameter route bernama `{activity}` dan controller menerima Activity `$activity`, Laravel dapat mencari model yang sesuai secara otomatis. Jika data tidak ditemukan, respons 404 dihasilkan. Ini mengurangi query berulang dan membuat method controller lebih jelas.

#### Contoh route model binding

```
public function edit(Activity $activity): View
{
```

```
return view('activities.edit', compact('activity'));
}
```

### 4.3 Form Request sebagai Boundary Input

#### Membuat Form Request

```
php artisan make:request StoreActivityRequest
php artisan make:request UpdateActivityRequest
```

#### StoreActivityRequest

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class StoreActivityRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'min:5', 'max:100'],
            'description' => ['nullable', 'string', 'max:1000'],
            'activity_date' => ['required', 'date'],
            'category' => ['required', 'string', 'max:50'],
            'status' => [
                'required',
                Rule::in(['Planned', 'Ongoing', 'Done']),
            ],
        ];
    }
}
```

#### Validation vs business rule

Form Request memeriksa apakah input layak masuk ke proses aplikasi: wajib/tidak, tipe, panjang, format, dan himpunan nilai yang diperbolehkan. Aturan seperti “Done tidak boleh kembali ke Planned” membutuhkan pengetahuan state sebelumnya dan merupakan business rule proses; aturan tersebut ditempatkan pada service.

### 4.4 Controller yang Fokus pada Orkestrasi

#### Contoh controller setelah pemisahan tanggung jawab

```
public function store(
    StoreActivityRequest $request,
    ActivityService $service
): RedirectResponse {
    $activity = $service->create($request->validated());

    return to_route('activities.show', $activity)
        ->with('success', 'Kegiatan berhasil dibuat.');
```

```
public function update(
    UpdateActivityRequest $request,
    Activity $activity,
    ActivityService $service
): RedirectResponse {
    try {
        $service->update($activity, $request->validated());
    } catch (DomainException $exception) {
        return back()
            ->withErrors(['status' => $exception->getMessage()])
            ->withInput();
    }

    return to_route('activities.show', $activity)
        ->with('success', 'Kegiatan berhasil diperbarui.');
```

## 4.5 Service Class dan Business Logic

Service class tidak dibuat karena “semua proyek Laravel harus memiliki service”. Service digunakan ketika terdapat logika aplikasi yang layak dipisahkan dari detail HTTP/controller. Pada kasus ini, aturan transisi status memberikan contoh konkret.

### app/Services/ActivityService.php — bagian 1

```
<?php

namespace App\Services;

use App\Models\Activity;
use DomainException;

class ActivityService
{
    private const TRANSITIONS = [
        'Planned' => ['Planned', 'Ongoing'],
        'Ongoing' => ['Ongoing', 'Done'],
        'Done' => ['Done'],
    ];

    public function create(array $data): Activity
    {
        return Activity::create($data);
    }

    public function update(Activity $activity, array $data): Activity
    {
        $nextStatus = $data['status'] ?? $activity->status;
        $this->ensureValidTransition($activity->status, $nextStatus);
        $activity->update($data);

        return $activity->refresh();
    }
}
```

### app/Services/ActivityService.php — bagian 2

```
private function ensureValidTransition(
    string $current,
    string $next
): void {
    $allowed = self::TRANSITIONS[$current] ?? [];

    if (! in_array($next, $allowed, true)) {
        throw new DomainException(
            "Transisi status {$current} ke {$next} tidak diizinkan."
        );
    }
}
}
```

**Pertanyaan pemahaman:** Mengapa rule min:5 lebih tepat berada pada Form Request, sedangkan transisi status lebih tepat dipisahkan sebagai business logic?

## 4.6 Form Blade dan Error Validation

### Potongan field form

```
<label for="title">Judul</label>
<input
    id="title"
    name="title"
    value="{{ old('title', $activity->title ?? '') }}"
>
@error('title')
    <p class="error">{{ $message }}</p>
@enderror

<select name="status" id="status">
    @foreach (['Planned', 'Ongoing', 'Done'] as $status)
        <option
            value="{{ $status }}"
            @selected(old('status', $activity->status ?? 'Planned') === $status)
        >
            {{ $status }}
        </option>
    @endforeach
</select>
```

```
@endforeach
</select>
@error('status')
    <p class="error">{{ $message }}</p>
@enderror
```

Blade menampilkan error yang dihasilkan server dan menjaga old input. Keputusan valid/tidak tetap dilakukan server. Tampilan tidak boleh menjadi satu-satunya mekanisme pembatasan nilai.

## 4.7 Clean Code dalam Konteks Laravel

| Aspek                 | Prinsip                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------|
| Penamaan              | Method store, update, destroy dan nama variabel harus menjelaskan tindakan/data.           |
| Method kecil          | Controller method berfokus pada orkestrasi, bukan mengandung banyak aturan bercabang.      |
| Single responsibility | Form Request, service, model, dan Blade memiliki alasan perubahan yang berbeda.            |
| Duplikasi             | Aturan yang sama tidak ditulis ulang di create.blade.php, edit.blade.php, dan controller.  |
| Guard / early return  | Kondisi tidak valid dapat dihentikan lebih awal agar alur utama mudah dibaca.              |
| Formatting            | Laravel Pint menjaga konsistensi gaya kode; ia bukan alat pembuktian desain atau perilaku. |
| Komentar              | Komentar menjelaskan alasan yang tidak jelas, bukan mengulang isi kode.                    |

### Menjalankan Laravel Pint

```
./vendor/bin/pint
```

## 4.8 Menulis Spesifikasi yang Dapat Dievaluasi

Mulai Modul 3, setiap fitur penting ditulis sebagai kondisi yang dapat diperiksa. Hindari spesifikasi seperti “fitur edit harus bagus” atau “CRUD berjalan”. Gunakan kondisi awal, aksi, hasil yang diharapkan, dan bukti.

| ID    | Precondition                | Aksi                                          | Expected Result                                                    |
|-------|-----------------------------|-----------------------------------------------|--------------------------------------------------------------------|
| AC-04 | Halaman create terbuka.     | Submit title = "Web" dengan field lain valid. | Data tidak tersimpan; pesan judul minimal 5 karakter tampil.       |
| AC-08 | Kegiatan berstatus Planned. | Update status menjadi Ongoing.                | Status tersimpan menjadi Ongoing.                                  |
| AC-09 | Kegiatan berstatus Done.    | Update status menjadi Planned.                | Update ditolak; status tetap Done; pesan transisi tidak diizinkan. |

## 4.9 Static Analysis dengan SonarQube

Static analysis memeriksa source code tanpa menjalankan seluruh skenario aplikasi. SonarQube digunakan pada modul ini sebagai alat observasi code smell, duplication, complexity, dan maintainability issue. Infrastruktur server sebaiknya disediakan dosen/laboratorium agar waktu praktikum tidak habis untuk administrasi server.

### Contoh sonar-project.properties

```
sonar.projectKey=modul3-nim
sonar.projectName=Modul 3 - NIM Nama
sonar.sources=app, routes, resources
sonar.exclusions=vendor/**, storage/**, bootstrap/cache/**, public/build/**
sonar.sourceEncoding=UTF-8
```

### Contoh menjalankan scanner

```
sonar-scanner -Dsonar.host.url=http://ALAMAT-SONARQUBE -Dsonar.token=TOKEN_MAHASISWA
```

### Cara membaca hasil

Jangan memperbaiki semua temuan secara mekanis. Pilih temuan yang dapat dijelaskan: apa masalahnya, mengapa relevan, bagian kode mana yang terpengaruh, perubahan apa yang dilakukan, dan apakah perilaku aplikasi tetap sesuai acceptance criteria setelah refaktorisasi.

## 5. Eksperimen Praktikum

### Eksperimen 1. Menelusuri Request Lifecycle Laravel

Tujuan eksperimen adalah membangun model mental request-response Laravel berdasarkan bukti yang dapat diamati. Mahasiswa tidak diminta menghafal semua class internal framework.

1. Pastikan server development berjalan dan buka DevTools Network.
2. Buka /activities dan catat method, status, URL, dan waktu response.
3. Jalankan php artisan route:list dan temukan route activities.index.
4. Tambahkan sementara logger pada method index controller, muat ulang halaman, lalu amati storage/logs/laravel.log.

#### Observasi sementara pada controller

```
logger()->info('ActivityController@index dipanggil');
```

12. Cari view yang dikembalikan controller dan tandai data yang diterima Blade.
13. Hapus logger sementara setelah bukti dicatat.

| Tahap                 | Prediksi | Bukti Aktual | Penjelasan |
|-----------------------|----------|--------------|------------|
| Browser → Route       |          |              |            |
| Route → Controller    |          |              |            |
| Controller → Eloquent |          |              |            |
| Controller → Blade    |          |              |            |
| Blade → Response      |          |              |            |

**Pertanyaan:** Pada tahap mana JavaScript Modul 2 mulai kembali berperan jika response Laravel berisi file JavaScript?

### Eksperimen 2. Mendiagnosis Fat Controller

Baca contoh berikut. Jangan langsung refactor. Tandai setiap bagian sebagai validasi input, business rule, persistence, presentation/response, atau orkestrasi.

#### Starter diagnosis

```
public function update(Request $request, Activity $activity)
{
    $request->validate([
        'title' => ['required', 'min:5', 'max:100'],
        'status' => ['required'],
    ]);

    if ($activity->status === 'Done' && $request->status === 'Planned') {
        return back()->withErrors([
            'status' => 'Status tidak boleh mundur.',
        ]);
    }

    if (! in_array($request->status, ['Planned', 'Ongoing', 'Done'])) {
        return back()->withErrors(['status' => 'Status tidak dikenal.']);
    }

    $activity->title = $request->title;
    $activity->status = $request->status;
    $activity->save();

    return redirect('/activities/'.$activity->id)
        ->with('success', 'Data diperbarui.');
```

| Potongan                     | Kategori Tanggung Jawab | Lokasi yang Lebih Tepat | Alasan |
|------------------------------|-------------------------|-------------------------|--------|
| \$request->validate(...)     |                         |                         |        |
| aturan status Done → Planned |                         |                         |        |
| in_array status              |                         |                         |        |

```
$activity->save()
```

redirect + flash

### Eksperimen 3. Business Rule di Tempat yang Salah

Bandingkan dua pendekatan: (A) Blade menyembunyikan pilihan status yang tidak diperbolehkan, tetapi server tetap menerima request; (B) Blade membantu UX, sementara service tetap menegakkan aturan. Uji request yang mencoba menurunkan status Done menjadi Planned.

#### Kesimpulan yang dicari

Perubahan tampilan dapat membantu pengguna, tetapi business rule harus ditegakkan pada proses server. UI tidak boleh menjadi satu-satunya penjaga aturan aplikasi. Prinsip ini akan menjadi lebih penting ketika authorization dipelajari pada Modul 5.

### Eksperimen 4. Static Analysis dan Refaktorisasi Terarah

1. Jalankan baseline SonarQube pada kondisi kode sebelum refactor final Task 3.
2. Pilih satu temuan yang benar-benar berada pada kode yang ditulis mahasiswa, bukan vendor/framework.
3. Catat kategori, lokasi, penjelasan tool, dan dugaan dampaknya terhadap maintainability.
4. Lakukan satu refaktorisasi kecil.
5. Jalankan kembali acceptance criteria yang terkait dan static analysis.

| Sebelum | Temuan | Keputusan Perbaikan | Sesudah / Bukti |
|---------|--------|---------------------|-----------------|
|---------|--------|---------------------|-----------------|

## 6. Tugas Praktikum

Task dikerjakan pada repository yang sama. Gunakan commit yang terpisah untuk setiap checkpoint agar evolusi desain dapat diperiksa. Minimal 70% fungsi utama pada task minggu tersebut harus selesai selama sesi praktikum.

| Komponen                         | Bobot | Hasil Utama                                                    |
|----------------------------------|-------|----------------------------------------------------------------|
| Task 1 – Laravel Skeleton        | 20%   | Request lifecycle, migration/model, index, detail.             |
| Task 2 – Activity CRUD           | 25%   | CRUD + Form Request + route model binding.                     |
| Task 3 – Business Logic Refactor | 25%   | Service + business rule + static analysis.                     |
| Homework + Checkpoint Integrasi  | 30%   | Refactor terarah, acceptance criteria, dokumentasi repository. |

### Task 1. Laravel Skeleton dan Request Lifecycle

#### Target

Membangun kerangka Activity Manager v1 yang dapat menampilkan daftar dan detail kegiatan dari database serta menjelaskan request lifecycle-nya.

#### Bahan Awal

#### Struktur target minimum Task 1

```
activity-manager/
├── app/
│   ├── Http/Controllers/ActivityController.php
│   └── Models/Activity.php
├── database/
│   ├── migrations/
│   └── seeders/ActivitySeeder.php
├── resources/views/
│   ├── layouts/app.blade.php
│   └── activities/
│       ├── index.blade.php
│       └── show.blade.php
├── routes/web.php
└── README.md
```

## Langkah Kerja

1. Buat proyek Laravel dan jalankan server. Catat versi PHP, Composer, dan Laravel pada README.
2. Konfigurasi database kelas dan jalankan migration bawaan tanpa error.
3. Buat model Activity dan migration activities sesuai struktur data Modul 3.
4. Buat seeder minimal lima kegiatan yang mencakup status Planned, Ongoing, dan Done.
5. Buat ActivityController dengan method index dan show.
6. Daftarkan route index dan show dengan nama route.
7. Buat layout Blade sederhana dan dua view: activities.index serta activities.show.
8. Jalankan route:list dan cocokkan route, controller method, serta URL.
9. Uji satu URL detail yang valid dan satu ID yang tidak tersedia.
10. Commit dengan pesan yang jelas, misalnya feat: add activity list and detail.

## Skenario Uji Wajib

| Skenario         | Tindakan                             | Expected Result                                  | Bukti             |
|------------------|--------------------------------------|--------------------------------------------------|-------------------|
| Server awal      | Jalankan php artisan serve.          | Aplikasi dapat diakses tanpa error 500.          | Network + halaman |
| Daftar           | GET /activities                      | Minimal lima kegiatan tampil.                    | Screenshot        |
| Detail valid     | Buka satu kegiatan.                  | Data yang dipilih tampil.                        | Screenshot + URL  |
| Detail tidak ada | Ubah ID menjadi data tidak tersedia. | HTTP 404.                                        | Network           |
| Route map        | php artisan route:list               | Route index/show menunjuk controller yang benar. | Terminal          |
| Data             | Periksa database/tinker.             | Seeder tersedia dan field sesuai migration.      | Bukti data        |

## Hint

- 1) Hint 1: Pastikan alur paling sederhana bekerja: route → controller → text response. Setelah itu baru sambungkan view.
- 2) Hint 2: Jika view tidak menerima data, gunakan dd(\$activities) sementara di controller, bukan menambah logika acak di Blade.
- 3) Hint 3: Jika route model binding menghasilkan 404, cocokkan nama parameter {activity} dengan type-hint Activity \$activity dan pastikan ID ada.

## Keluaran dan Bukti Wajib

- 1) Repository GitHub dapat diakses dan commit Task 1 dapat ditunjukkan.
- 2) Screenshot halaman daftar dan detail.
- 3) Output route:list yang relevan.
- 4) Bukti migration dan seeder berhasil.
- 5) Diagram request lifecycle untuk satu request GET /activities.
- 6) Jawaban worksheet yang menjelaskan fungsi route, controller, Eloquent, dan Blade.

## Task 2. Activity CRUD dengan Form Request

### Target

Melengkapi create, store, edit, update, dan destroy menggunakan resource controller, route model binding, Form Request, error validation, redirect, dan flash message.

## Bahan Awal

### Struktur file Task 2

```
app/Http/Controllers/ActivityController.php
app/Http/Requests/StoreActivityRequest.php
app/Http/Requests/UpdateActivityRequest.php
resources/views/activities/
├── _form.blade.php
├── create.blade.php
├── edit.blade.php
├── index.blade.php
├── show.blade.php
routes/web.php
```

**Starter method resource controller**

```

public function create(): View
{
    return view('activities.create');
}

public function store(StoreActivityRequest $request): RedirectResponse
{
    // TODO: simpan $request->validated()
    // TODO: redirect ke detail + flash message
}

public function edit(Activity $activity): View
{
    return view('activities.edit', compact('activity'));
}

public function update(
    UpdateActivityRequest $request,
    Activity $activity
): RedirectResponse {
    // TODO: update data tervalidasi
    // TODO: redirect ke detail + flash message
}

public function destroy(Activity $activity): RedirectResponse
{
    // TODO: hapus satu model
    // TODO: redirect ke daftar
}

```

**Starter resources/views/activities/\_form.blade.php**

```

<label for="title">Judul</label>
<input
    id="title"
    name="title"
    value="{{ old('title', $activity->title ?? '') }}"
>
@error('title')
    <p class="error">{{ $message }}</p>
@enderror

{{-- TODO: description, activity_date, category, status --}}

```

**Langkah Kerja**

1. Ubah route menjadi resource route dan verifikasi route:list.
2. Buat StoreActivityRequest serta UpdateActivityRequest.
3. Implementasikan rules BR-01 sampai BR-03 pada Form Request.
4. Buat view create dan edit. Hindari duplikasi markup dengan partial form bila duplikasi mulai jelas.
5. Implementasikan store menggunakan \$request->validated().
6. Implementasikan edit dan update menggunakan route model binding.
7. Implementasikan destroy dan konfirmasi aksi pada UI.
8. Tampilkan error dekat input menggunakan directive @error.
9. Pertahankan old input setelah validasi gagal.
10. Uji semua acceptance criteria CRUD sebelum melanjutkan ke Task 3.

**Skenario Uji Wajib**

| ID    | Input / Aksi       | Expected Result                             | Bukti            |
|-------|--------------------|---------------------------------------------|------------------|
| T2-01 | Tambah data valid. | Satu data baru tersimpan.                   | UI + DB          |
| T2-02 | Title 4 karakter.  | Ditolak; title error; data tidak bertambah. | UI               |
| T2-03 | Status = Unknown.  | Ditolak server.                             | UI / response    |
| T2-04 | Tanggal kosong.    | Ditolak; pesan wajib diisi.                 | UI               |
| T2-05 | Ubah title valid.  | Data lama berubah tanpa duplikasi.          | UI + DB          |
| T2-06 | Hapus satu data.   | Hanya data target yang hilang.              | UI + jumlah data |

| ID    | Input / Aksi            | Expected Result                         | Bukti       |
|-------|-------------------------|-----------------------------------------|-------------|
| T2-07 | Refresh sesudah submit. | Tidak membuat data tambahan.            | Jumlah data |
| T2-08 | Keyboard form.          | Kontrol utama dapat dicapai dan submit. | Catatan uji |

### Hint

1. Hint 1: Selesaikan create/store sampai lulus seluruh skenario validasi sebelum mengerjakan edit/update.
2. Hint 2: Gunakan partial \_form.blade.php untuk field yang sama pada create dan edit, tetapi jangan memindahkan validasi server ke partial tersebut.
3. Hint 3: Bila update menghasilkan data ganda, periksa apakah method update memanggil create() atau mengubah model hasil route model binding.

### Keluaran dan Bukti Wajib

1. File resource controller, dua Form Request, dan Blade create/edit/form partial.
2. Tabel delapan skenario uji pada worksheet.
3. Tangkapan layar validation error dan satu submit valid.
4. Output route:list untuk resource activities.
5. Commit terpisah untuk CRUD sebelum refactor Task 3.

### Kriteria Selesai

1. Form Request digunakan untuk create dan update; validation rules tidak diduplikasi di Blade.
2. CRUD berjalan melalui resource route/controller.
3. Route model binding digunakan pada show/edit/update/destroy.
4. Validation error dan old input dapat diamati.
5. Tidak ada query database yang ditulis langsung pada Blade.
6. Tidak ada error aplikasi yang belum dijelaskan pada Console/Network/log pada alur normal.

## Task 3. Business Logic Refactor dan Quality Baseline

### Target

Memisahkan aturan transisi status ke ActivityService, menjaga controller tetap fokus pada orkestrasi, menulis acceptance criteria, dan menggunakan SonarQube untuk menilai satu perubahan kualitas kode.

### Bahan Awal

#### Starter ActivityService

```
<?php

namespace App\Services;

use App\Models\Activity;

class ActivityService
{
    public function create(array $data): Activity
    {
        // TODO
    }

    public function update(Activity $activity, array $data): Activity
    {
        // TODO: periksa transisi status
        // TODO: update dan return model
    }

    private function ensureValidTransition(
        string $current,
        string $next
    ): void {
        // TODO: aturan BR-03A
    }
}
```

#### Starter controller setelah service diperkenalkan

```

public function update(
    UpdateActivityRequest $request,
    Activity $activity,
    ActivityService $service
): RedirectResponse {
    // TODO: panggil service
    // TODO: tangani DomainException sebagai validation feedback
    // TODO: redirect saat berhasil
}

```

### Langkah Kerja

1. Tuliskan terlebih dahulu aturan transisi status dan tabel expected result pada worksheet.
2. Jalankan baseline static analysis atau baseline code review sebelum refactor.
3. Buat app/Services/ActivityService.php.
4. Pindahkan aturan transisi status dari controller/Blade ke service.
5. Pastikan create/update controller memanggil service dan tetap menangani response HTTP.
6. Uji transisi Planned → Ongoing dan Ongoing → Done.
7. Uji negative case Done → Ongoing dan Ongoing → Planned.
8. Jalankan kembali CRUD normal case untuk memastikan refactor tidak merusak fitur sebelumnya.
9. Jalankan Laravel Pint.
10. Jalankan SonarQube, pilih satu temuan relevan, lakukan satu perbaikan nyata, lalu dokumentasikan sebelum/sesudah.

### Skenario Uji Wajib

| Skenario       | Current | Next    | Expected  |
|----------------|---------|---------|-----------|
| Tetap Planned  | Planned | Planned | Diterima. |
| Mulai kegiatan | Planned | Ongoing | Diterima. |
| Selesai        | Ongoing | Done    | Diterima. |
| Tetap Done     | Done    | Done    | Diterima. |
| Mundur 1       | Done    | Ongoing | Ditolak.  |
| Mundur 2       | Done    | Planned | Ditolak.  |
| Mundur 3       | Ongoing | Planned | Ditolak.  |

### Hint

1. Hint 1: Tulis matriks current → next terlebih dahulu sebelum menulis if atau array transition.
2. Hint 2: Service perlu mengetahui current status dari model dan next status dari data update. Jangan bergantung pada input Blade saja.
3. Hint 3: Bila controller mulai panjang karena penanganan error, pastikan yang dipindahkan adalah business logic, bukan seluruh detail HTTP secara membabi buta.

### Bukti Static Analysis

| Elemen            | Yang Dicatat                                                                     |
|-------------------|----------------------------------------------------------------------------------|
| Baseline          | Tanggal scan, project key, jumlah/jenis temuan yang relevan pada kode mahasiswa. |
| Temuan dipilih    | Rule/issue, file/baris, alasan dianggap relevan.                                 |
| Perubahan         | Refactor yang dilakukan dan alasan.                                              |
| Verifikasi fungsi | Acceptance criteria yang dijalankan ulang.                                       |
| Hasil akhir       | Status issue atau perubahan metric setelah scan ulang.                           |

### Keluaran dan Bukti Wajib

1. ActivityService dengan aturan transisi status yang dapat dijelaskan.
2. Controller setelah refactor dan perbandingan tanggung jawab sebelum/sesudah.
3. Tabel skenario transisi status normal dan negative case.
4. Bukti scan SonarQube sebelum/sesudah satu perbaikan.
5. Bukti acceptance criteria Task 2 tetap lulus setelah refactor.

6. Commit terpisah, misalnya refactor: move activity status rule to service.

## 7. Independent Challenge dan Homework

### 7.1 Independent Challenge – Filter Kegiatan

Tambahkan filter status atau kategori pada halaman daftar tanpa mengubah data asli. Filter dikirim sebagai query string sehingga URL merepresentasikan state filter, misalnya /activities?status=Planned.

- Filter “Semua” menampilkan seluruh data.
- Filter status menampilkan subset yang benar.
- Nilai filter tidak valid tidak menyebabkan error server; tentukan perilaku yang jelas.
- Filter tetap terlihat setelah halaman dimuat ulang karena nilai berasal dari query string.
- Query database tetap berada pada layer aplikasi/controller/query scope, bukan pada Blade.
- Tuliskan minimal empat acceptance criteria sebelum implementasi.

#### Hint

Mulai dari `Activity::query()`, kemudian tambahkan kondisi dengan `when(...)` hanya jika parameter filter valid. Bila query mulai digunakan berulang pada modul selanjutnya, pertimbangkan local scope; jangan membuat abstraction sebelum ada kebutuhan yang jelas.

### 7.2 Homework – Architecture Review dan Refactor Terarah

- Pilih satu bagian kode yang masih memiliki duplikasi, method terlalu panjang, penamaan tidak jelas, atau tanggung jawab bercampur.
- Tuliskan masalahnya terlebih dahulu pada worksheet beserta bukti lokasi file/baris.
- Lakukan refactor kecil tanpa menambah fitur baru.
- Jalankan ulang acceptance criteria yang terkait.
- Bandingkan satu bukti sebelum dan sesudah, dapat berupa potongan kode, SonarQube, atau struktur method.
- Tuliskan satu keputusan arsitektur: mengapa logic tersebut ditempatkan pada Form Request, service, controller, model, atau Blade.
- Perbarui README dengan cara menjalankan proyek, setup database, seed, dan URL route utama.

## 8. Checkpoint Integrasi Modul 3 – Activity Manager v1

Checkpoint bukan mini project baru. Ini adalah kondisi repository yang harus siap digunakan sebagai baseline Modul 4. Seluruh fitur Modul 3 berada pada satu aplikasi yang konsisten dan dapat diperiksa kembali melalui acceptance criteria.

### 8.1 Definition of Done

1. Proyek menggunakan Laravel 13 dan dapat dijalankan dari clone baru dengan instruksi README.
2. Migration dan seeder dapat dijalankan tanpa edit manual file vendor.
3. CRUD kegiatan bekerja pada normal case.
4. Form Request menegakkan BR-01 sampai BR-03.
5. ActivityService menegakkan aturan transisi status BR-03A.
6. Route model binding digunakan pada resource yang relevan.
7. Blade tidak melakukan query database dan tidak menyimpan business rule utama.
8. Acceptance criteria AC-01 sampai AC-10 memiliki status dan bukti.
9. Pint telah dijalankan.
10. SonarQube baseline tersedia dan minimal satu keputusan refactor didokumentasikan.
11. Repository memiliki commit history yang dapat dibaca dan tag/checkpoint modul, misalnya modul-3-final.
12. Worksheet Modul 3 telah diisi dan URL repository tercantum.

### 8.2 Struktur Repository Minimum

Struktur minimum checkpoint

```
activity-manager/
├── app/
│   ├── Http/
│   │   ├── Controllers/ActivityController.php
│   │   └── Requests/
│   │       └── StoreActivityRequest.php
```

```

├── UpdateActivityRequest.php
├── Models/Activity.php
├── Services/ActivityService.php
├── database/
│   ├── migrations/
│   └── seeders/ActivitySeeder.php
├── resources/views/
│   ├── layouts/app.blade.php
│   └── activities/
├── routes/web.php
├── sonar-project.properties
└── README.md

```

## 8.3 Checklist Repository dan GitHub

1. .env tidak dikomit.
2. .env.example berisi contoh konfigurasi yang diperlukan.
3. vendor/ tidak dikomit.
4. Readme menjelaskan composer install, setup .env, key:generate, migrate --seed, dan serve.
5. URL repository dapat diakses dosen.
6. Commit tidak hanya satu commit besar di akhir; perubahan utama memiliki checkpoint yang dapat ditelusuri.

## 9. Penilaian dan Refleksi

### 9.1 Self Assessment

Gunakan skala 1–5: 1 belum mampu; 2 mampu dengan banyak bantuan; 3 mampu dengan panduan; 4 mandiri; 5 mampu menjelaskan kepada orang lain.

| Pernyataan | Skor 1–5 |
|------------|----------|
|------------|----------|

Saya dapat menjelaskan request lifecycle Laravel dari request sampai response.

Saya dapat membedakan route, controller, Form Request, service, model, migration, dan Blade.

Saya dapat membuat migration dan membaca/mengubah data melalui Eloquent.

Saya dapat menggunakan resource route/controller dan route model binding.

Saya dapat menjelaskan perbedaan validation rule dan business rule.

Saya dapat memindahkan business logic keluar dari Blade/controller jika tanggung jawabnya tidak tepat.

Saya dapat menulis acceptance criteria yang dapat diuji.

Saya dapat membaca temuan SonarQube dan memutuskan apakah temuan tersebut relevan.

Saya dapat menjelaskan seluruh kode yang saya kumpulkan dan mengubah bagian kecil tanpa AI.

### 9.2 Peer Assessment

| Kriteria         | Fokus Pemeriksaan                                                    |
|------------------|----------------------------------------------------------------------|
| Alur request     | Penilai dapat menelusuri route → controller → service/model → Blade. |
| CRUD             | Normal case dan validasi gagal dapat direproduksi.                   |
| Business logic   | Aturan status tidak hanya berada pada UI.                            |
| Clean code       | Penamaan dan pemisahan tanggung jawab dapat dibaca.                  |
| Repository       | README, commit, dan struktur folder dapat dipahami.                  |
| Quality evidence | Acceptance criteria dan static analysis memiliki bukti.              |

### 9.3 Lesson Learnt

1. Komponen Laravel mana yang sebelumnya terasa “magis” tetapi sekarang dapat Anda jelaskan alurnya?
2. Apa perbedaan paling penting antara validation rule dan business rule pada modul ini?

3. Bagian controller mana yang paling layak dipindahkan dan mengapa?
4. Apakah semua temuan SonarQube harus diperbaiki? Jelaskan keputusan Anda menggunakan satu contoh.
5. Keputusan arsitektur apa yang kemungkinan paling membantu ketika aplikasi mulai memiliki authentication dan role?
6. Kesalahan apa yang paling membantu Anda memahami route model binding atau persistence?
7. Kompetensi apa yang perlu diperkuat sebelum Modul 4?

## 10. Rubrik Penilaian Dosen

| Kriteria                                                                                                                                                                                                                                                                 | Skor Rendah                                            |                                       | Skor Cukup                                                     | Skor Tinggi                                                               | Bobot                                                                                     |     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----|
| Laravel lifecycle                                                                                                                                                                                                                                                        | Tidak dapat menelusuri request.                        |                                       | Route/controller/view bekerja tetapi penjelasan masih parsial. | Menjelaskan alur request-response dan bukti setiap titik utama.           | 15%                                                                                       |     |
| Persistence & CRUD                                                                                                                                                                                                                                                       | Migration/model CRUD tidak stabil.                     | atau                                  | CRUD normal case bekerja.                                      | CRUD, route model binding, validation, dan negative case teruji.          | 20%                                                                                       |     |
| Business architecture                                                                                                                                                                                                                                                    | logic                                                  | Logic tersebar pada Blade/controller. |                                                                | Sebagian tanggung jawab terpisah.                                         | Form Request, service, model, controller, dan Blade memiliki batas yang dapat dijelaskan. | 20% |
| Clean code                                                                                                                                                                                                                                                               | Duplikasi dan method besar tidak dijelaskan.           |                                       | Kode terbaca dan format konsisten.                             | Refactor terarah memperbaiki maintainability tanpa merusak perilaku.      | 15%                                                                                       |     |
| Spesifikasi & bukti                                                                                                                                                                                                                                                      | Tidak ada acceptance criteria atau bukti.              |                                       | Sebagian acceptance criteria diuji.                            | Normal, boundary/negative case dan bukti terhubung dengan business rules. | 15%                                                                                       |     |
| Static analysis                                                                                                                                                                                                                                                          | Tidak dijalankan atau hanya screenshot tanpa analisis. |                                       | Hasil tersedia dan satu temuan dijelaskan.                     | Temuan relevan diprioritaskan, diperbaiki, lalu diverifikasi ulang.       | 10%                                                                                       |     |
| Proses & pemahaman                                                                                                                                                                                                                                                       | Tidak mampu menjelaskan kode.                          |                                       | Menjelaskan bantuan.                                           | dengan Menjelaskan dan mengubah kode secara mandiri.                      | 5%                                                                                        |     |
| <b>Prinsip penilaian</b><br>Nilai akhir mempertimbangkan kebenaran fungsi, pemisahan tanggung jawab, bukti pengujian, dan kemampuan menjelaskan keputusan. Tampilan visual yang menarik tidak dapat menggantikan business rule yang salah atau kode yang tidak dipahami. |                                                        |                                       |                                                                |                                                                           |                                                                                           |     |

## 11. Referensi

1. Laravel 13 Documentation. Request Lifecycle. <https://laravel.com/docs/13.x/lifecycle>
2. Laravel 13 Documentation. Installation and Configuration. <https://laravel.com/docs/13.x>
3. Laravel 13 Documentation. Routing. <https://laravel.com/docs/13.x/routing>
4. Laravel 13 Documentation. Controllers. <https://laravel.com/docs/13.x/controllers>
5. Laravel 13 Documentation. Blade Templates. <https://laravel.com/docs/13.x/blade>
6. Laravel 13 Documentation. Validation and Form Requests. <https://laravel.com/docs/13.x/validation>
7. Laravel 13 Documentation. Migrations. <https://laravel.com/docs/13.x/migrations>
8. Laravel 13 Documentation. Eloquent ORM. <https://laravel.com/docs/13.x/eloquent>
9. Laravel 13 Documentation. Laravel Pint. <https://laravel.com/docs/pint>
10. SonarSource. SonarQube Community Build / Server documentation and PHP analysis. <https://docs.sonarsource.com/sonarqube-community-build/>
11. Robert C. Martin. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
12. Martin Fowler. Refactoring: Improving the Design of Existing Code. Addison-Wesley.